EECS 348 — Assignment 5

1. Programs and access

Google Gemini, accessed through gemini.google.com and Anthropic Claude, accessed through claude.ai. The same original Assignment 4 source by Jacob Dean Lehrman was provided to both systems.

2. Identical prompt

The following prompt was provided to both systems:

I am completing EECS 348 Software Engineering Assignment 5 at the University of Kansas. I have attached another student's Assignment 4 C++ Sudoku solver.

I need you to modify the existing program to meet the following requirements. Requirements:

1. Preserve the original student's code structure, SudokuSolver class, recursive backtracking algorithm, and existing functionality. Do not completely rewrite the program.
2. Modify the program so that it reads five Sudoku puzzle files named puzzle1.txt through puzzle5.txt.
3. For each puzzle, create a corresponding output file named solution1.txt through solution5.txt.
4. Each output file must contain every valid Sudoku solution found by the recursive backtracking algorithm.
5. Format each solution as a 9x9 grid of digits separated by spaces, matching the input puzzle format.
6. If solutions are found, include the exact line "Solutions found: n" at the end of the output file, where n is the total number of solutions.
7. If no solutions exist, the output file must contain only the line "No solution found."
8. Maintain object-oriented programming principles and use recursive depth-first search with backtracking.
9. Include descriptive comments explaining the modifications, functions, and major sections of code.
10. Preserve the original author's information in the prologue comments. Add a placeholder for "Revised by:" and the revision date. Identify any AI-generated modifications in the comments.
11. Ensure the program compiles using g++ on a Linux server with C++17.
12. Include appropriate error handling for missing or invalid puzzle files. Important constraints: Modify the provided Assignment 4 code rather than creating an entirely new implementation. Preserve as much of the original student's code as reasonably possible. Keep everything in one C++ source file named Assignment5.cpp. Do not use external libraries beyond the C++ standard library. Do not create a Makefile or additional source files. Output requested: First, provide the

complete modified Assignment5.cpp source code. After the code, briefly explain which sections of the original program were changed and why. Do not omit any source code or replace sections with placeholders.

3. Correctness and testing

Both unmodified AI-generated programs compiled on cycle server and completed all five supplied puzzle files. Observed counts: puzzle1=1, puzzle2=1, puzzle3=1, puzzle4=0, puzzle5=2. Both write the specified count line for puzzles with solutions and the required no-solution message for puzzle4.

4. Execution time and space complexity

Each implementation uses depth-first recursive backtracking. For E empty cells the worst case search explores O(9^E) assignments, with O(81) board storage and O(81) recursive stack depth (excluding output). Both stream solutions to files, so memory usage does not grow with the number of solutions. Claude adds constant-size validation work per cell and an extra 9x9 temporary board. The observed local runtimes were approximately 0.016 s for Claude and 0.019 s for Gemini; these single run measurements are too small and noisy to establish a reliable performance winner.

5. Maintainability and reliability

Claude separates plain grid serialization into writeBoard(), checks conflicting starting clues, validates each input character, and preserves original console output. Gemini writes grids directly inside the recursive base case, checks only whether the input file opened, and removes the console reporting from Assignment 4. Claude has clearer change attribution tags, while Gemini is shorter and passes its output stream as an explicit parameter.

6. Selection and justification

Claude was selected because it provides stronger validation, preserves original functionality, and makes solution file formatting reusable. Both versions correctly handled the five supplied puzzles, so the choice is primarily about reliability on malformed inputs and maintainability rather than proven speed.

7. Improvements made to selected version

The final Assignment5.cpp adds rejection of extra non-whitespace input after the expected 81 cells. This closes a validation gap where malformed files with extra data were silently accepted. It also explicitly clears the reusable output-stream state before opening the next file. These changes do not alter worst-case backtracking time or asymptotic memory complexity. The revision-date placeholder was completed. Original class and recursive search structure were Retained.

8. Complete raw AI-generated source code

Claude — complete raw generated code

```
/*
Name: EECS348 Assignment 4 (Revised for Assignment 5)
Description: Code to solve Sudoku Puzzles in a depth-first manner with backtracking.
             Revised for Assignment 5 so that every solution found is also written to a
             solution file (solution1.txt - solution5.txt), one per puzzle file.
Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
Outputs: Initial (unsolved) puzzle, all solved puzzles, total number of solutions found
(console),
         solution1.txt, solution2.txt, solution3.txt, solution4.txt, solution5.txt (files).
Collaborators: None
Other Sources: All code generated by Google Gemini (original Assignment 4 code).
               Assignment 5 modifications generated by Claude (AI, Anthropic).
Author: Jacob Dean Lehrman
Creation Date: 15/10/2026 (dd/mm/yyyy)
Revised by: Peter Lauman
Revision Date: [PLACEHOLDER - dd/mm/yyyy]   (modifications drafted 29/10/2026)

AI DISCLOSURE: Every comment tagged "[A5 AI-MODIFIED]" or "[A5 AI-ADDED]" marks code that was
changed or added by an AI assistant (Claude) for Assignment 5. All other code is the original
Assignment 4 code and is left unchanged.
*/

//Declare utilizations of necessary libraries.
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {//Beginning the overall class
private://Variables and functions not accessible outside the class.
    std::vector<std::vector<char>> board;//Placeholder 2-D array of undetermined size to
represent the board.

    // [A5 AI-ADDED] Output stream for the current puzzle's solution file. Every solution
found by the
    // recursive search is written to this stream as soon as it is discovered.
    std::ofstream outFile;

    // Check if placing character 'c' at board[row][col] is valid
    bool isValid(int row, int col, char c) const {//Function to see if it is possible to
place a value at a particular coordinate
        for (int i = 0; i < 9; ++i) {//Since both the row and collum are the same size and
both need to be checked, can use one loop for both
            // Check row and column
            if (board[row][i] == c) return false;//If the row already contains the value, it
cant be placed there and further execution is not needed.
            if (board[i][col] == c) return false;//If the collum alrady contains the value,
it can't be placed there and the function can stop.
        }

        // Check 3x3 subgrid
        int startRow = row - row % 3;//Since the squares are 3x3, this line will return the
row corresonding to the top of the square.
```

```
        int startCol = col - col % 3;//Since the squares are 3x3, this line will return the
collum corresponding to the left of the square.
        for (int i = 0; i < 3; ++i) {//Iterate through the rows in the square
            for (int j = 0; j < 3; ++j) {//Iterate through the collums in the square
                if (board[startRow + i][startCol + j] == c) return false;//If the value is in
the square, it can't be placed there and execution stops.
            }
        }
        return true;//If it passes all previous checks, then the value is valid and can be
placed there.
    }

    // [A5 AI-ADDED] Checks that the clues supplied in the puzzle do not already contradict
each other
    // (e.g., two 5s in one row). The recursive solver skips pre-filled cells without
validating them,
    // so without this check an inconsistent puzzle could be reported as "solved."
    // Each clue is temporarily removed and re-tested with the original isValid() function.
    bool initialBoardIsConsistent() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (board[i][j] != '_') {//Only pre-filled cells need to be checked.
                    char c = board[i][j];//Remember the clue.
                    board[i][j] = '_';//Temporarily blank it so it does not conflict with
itself.
                    bool ok = isValid(i, j, c);//Is the clue legal given every other clue?
                    board[i][j] = c;//Restore the clue.
                    if (!ok) return false;//A contradiction means the puzzle has no solution.
                }
            }
        }
        return true;//No contradictions found.
    }

    // [A5 AI-ADDED] Writes the board to an output stream in the same plain format as the
input files:
    // nine rows of nine characters separated by single spaces, with no divider lines.
    void writeBoard(std::ostream& out) const {
        for (int i = 0; i < 9; ++i) {//Iterate through the rows
            for (int j = 0; j < 9; ++j) {//Iterate through the collums of each row
                out << board[i][j];//Write the cell value.
                if (j < 8) out << " ";//Separate values with a single space (no trailing
space).
            }
            out << "\n";//End the row.
        }
    }

    // Recursive DFS backtracking function to find all solutions
    bool solveRecursive(int row, int col, int& solutionCount) {
        // If we have crossed the last row, a complete solution is found
        if (row == 9) {//Since the vectors are 0-indexed, 9 is out of bounds and reachign it
means that all cells have been successfully filled.
            solutionCount++;//All cells filled means the puzzle is solved, so increment the
counter.
            //Pritns the solved board with a clear header indiacting which solution it is.
            std::cout << "Solution " << solutionCount << ":\n";
            printBoard();
            std::cout << "\n";
```

```
            // [A5 AI-ADDED] Also write this solution to the solution file (plain 9x9 grid),
            // followed by a blank line to separate it from the next solution.
            writeBoard(outFile);
            outFile << "\n";
            return false; // Return false to force backtracking and search for alternative
solutions
        }

        // Move to the next row if column reaches 9
        if (col == 9) {//Since the valid range of address is 0-8, 9 means that the collums in
the row have been successfully filled.
            return solveRecursive(row + 1, 0, solutionCount);//Go to the next row.
        }

        // If cell is already filled, skip to the next column
        if (board[row][col] != '_') {
            return solveRecursive(row, col + 1, solutionCount);
        }

        // Try digits '1' through '9'
        for (char c = '1'; c <= '9'; ++c) {//Hacky solutions takign advantage of ASCII
characters
            if (isValid(row, col, c)) {//If the value can be placed...
                board[row][col] = c;//Place that value
                solveRecursive(row, col + 1, solutionCount);//Recursively check the next cell
in sequence
                board[row][col] = '_'; // Backtrack
            }
        }

        return false;//If the code reaches this point, it has checked all possible
combinations.
    }

public:
    SudokuSolver() : board(9, std::vector<char>(9, '_')) {}//Formally declare the 2-D array
to be 9x9 filled with _.

    // Load puzzle from file
    // [A5 AI-MODIFIED] Original only checked that the file opened. It now also validates the
contents:
    // exactly 81 cell values, each either '1'-'9' or '_'. The board is only replaced if the
whole
    // file is valid, so a bad file never leaves a half-loaded board behind.
    // Returns false (and fills errorMessage) on a missing/unreadable/invalid file.
    bool loadFromFile(const std::string& filename, std::string& errorMessage) {
        std::ifstream file(filename);//open a file of the given name.
        if (!file.is_open()) {//If the file fails to open...
            errorMessage = "Could not open file " + filename + " (missing or unreadable).";
            return false;//Indicate that the file couldn't be loaded.
        }
        std::vector<std::vector<char>> temp(9, std::vector<char>(9, '_'));//[A5 AI-ADDED]
Load into a temporary board first.
        for (int i = 0; i < 9; ++i) {//Iterate through all 81 possible cells in the array
            for (int j = 0; j < 9; ++j) {
                char c;
                if (!(file >> c)) {//[A5 AI-ADDED] The file ended before 81 values were read.
                    errorMessage = "File " + filename + " is invalid: it contains fewer than
81 cell values.";
```

```
                    return false;
                }
                if (!((c >= '1' && c <= '9') || c == '_')) {//[A5 AI-ADDED] Reject any
unexpected character.
                    errorMessage = std::string("File ") + filename + " is invalid: unexpected
character '" + c +
                                   "' (row " + std::to_string(i + 1) + ", column " +
std::to_string(j + 1) +
                                   "). Only digits 1-9 and '_' are allowed.";
                    return false;
                }
                temp[i][j] = c;//Iteratively writes non-whitespace characters into the 2-D
array
            }
        }
        board = temp;//[A5 AI-ADDED] Only now is the real board replaced.
        return true;//Indiacte that the puzzle has been loaded into the program successfully.
    }

    // Print the Sudoku board with clear grid formatting
    void printBoard() const {
        for (int i = 0; i < 9; ++i) {//Iterate through the rows
            if (i > 0 && i % 3 == 0) {//Only print the row divider after 3 rows have been
printed.
                std::cout << "------+-------+------\n";
            }
            for (int j = 0; j < 9; ++j) {//Iterate through the collums of each row
                if (j > 0 && j % 3 == 0) {//Only print the collum divider after three values
fro mthe row have been printed.
                    std::cout << "| ";
                }
                std::cout << board[i][j] << " ";//Print the value with trailing whitespace to
make it easier to read.
            }
            std::cout << "\n";//Go to the next row, visually.
        }
    }

    // Driver method to process a puzzle file
    // [A5 AI-MODIFIED] Now takes the name of the solution file as a second argument, opens
it,
    // and writes either all solutions plus "Solutions found: n" or only "No solution found."
    void solvePuzzleFile(const std::string& filename, const std::string& outputFilename) {
        //Prints the puzzle file solved
        std::cout << "====================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "====================================\n";

        std::string errorMessage;//[A5 AI-ADDED] Describes why a puzzle could not be loaded.
        if (!loadFromFile(filename, errorMessage)) {//If the file can't be opened or is
invalid, indicate that an error has occured.
            // [A5 AI-MODIFIED] Message now explains the specific problem. The program
reports the
            // error and moves on to the next puzzle instead of crashing.
            std::cout << "Error: " << errorMessage << "\n\n";
            std::cerr << "Error: " << errorMessage << "\n";
            return;
        }
        //Print the initially supplied board, with formatting, to meet requirements.
```

```
        std::cout << "Original Puzzle:\n";
        printBoard();
        std::cout << "\n";

        // [A5 AI-ADDED] Open the solution file (overwriting any previous contents).
        outFile.open(outputFilename);
        if (!outFile.is_open()) {//If the output file cannot be created (e.g., no write
permission)...
            std::cout << "Error: Could not create output file " << outputFilename << ".\n\n";
            std::cerr << "Error: Could not create output file " << outputFilename << ".\n";
            return;
        }

        int solutionCount = 0;//Counter to hold the number of possible ways the puzzle can be
solved.

        // [A5 AI-ADDED] Only search if the supplied clues do not contradict each other.
        // Otherwise solutionCount stays 0 and the "no solution" branch below handles it.
        if (initialBoardIsConsistent()) {
            solveRecursive(0, 0, solutionCount);//Solve the puzzle, starting at the top-left
square.
        }

        if (solutionCount == 0) {//If no solutions, inform the user
            std::cout << "No solution found.\n\n";
            // [A5 AI-ADDED] The file must contain ONLY this line. Nothing was written during
the
            // search because no solution was found, so the file is still empty here.
            outFile << "No solution found." << "\n";
        } else {//If at least one solution, indicate how many.
            std::cout << "Total solutions found: " << solutionCount << "\n\n";
            // [A5 AI-ADDED] Required final line of the solution file.
            outFile << "Solutions found: " << solutionCount << "\n";
        }

        outFile.close();//[A5 AI-ADDED] Flush and close the solution file before the next
puzzle.
    }
};

int main() {//Start of actually running the program
    SudokuSolver solver;//Instantiate the Solver class

    // Process puzzle1.txt through puzzle5.txt
    for (int i = 1; i <= 5; ++i) {//Since the puzzles have a consistent naming convention,
their filenames can be easily iterated over with simple concatenation.
        std::string filename = "puzzle" + std::to_string(i) + ".txt";
        // [A5 AI-ADDED] The matching output file uses the same number: puzzleN.txt ->
solutionN.txt
        std::string outputFilename = "solution" + std::to_string(i) + ".txt";
        solver.solvePuzzleFile(filename, outputFilename);// [A5 AI-MODIFIED] Passes the
output file name too.
    }

    return 0;//Default return value.
}//End main
```

Gemini — complete raw generated code

```
/*
Name: EECS348 Assignment 5 (Modified from Assignment 4)
Description: Code to solve Sudoku Puzzles in a depth-first manner with backtracking,
outputting results to solution files.
Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
Outputs: solution1.txt, solution2.txt, solution3.txt, solution4.txt, solution5.txt
Collaborators: None
Other Sources: Original Assignment 4 code by Jacob Dean Lehrman; modifications for file
output, formatting, and error handling assisted by AI (Google Gemini).
Author: Jacob Dean Lehrman
Creation Date: 15/10/2026 (dd/mm/yyyy)
Revised by: Peter Lauman
Revision Date: 29/10/2026 (dd/mm/yyyy)
*/

// Declare utilizations of necessary libraries.
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver { // Beginning the overall class
private: // Variables and functions not accessible outside the class.
    std::vector<std::vector<char>> board; // Placeholder 2-D array of undetermined size to
represent the board.

    // Check if placing character 'c' at board[row][col] is valid
    bool isValid(int row, int col, char c) const { // Function to see if it is possible to
place a value at a particular coordinate
        for (int i = 0; i < 9; ++i) { // Since both the row and column are the same size and
both need to be checked, can use one loop for both
            // Check row and column
            if (board[row][i] == c) return false; // If the row already contains the value,
it can't be placed there and further execution is not needed.
            if (board[i][col] == c) return false; // If the column already contains the
value, it can't be placed there and the function can stop.
        }

        // Check 3x3 subgrid
        int startRow = row - row % 3; // Since the squares are 3x3, this line will return the
row corresponding to the top of the square.
        int startCol = col - col % 3; // Since the squares are 3x3, this line will return the
column corresponding to the left of the square.
        for (int i = 0; i < 3; ++i) { // Iterate through the rows in the square
            for (int j = 0; j < 3; ++j) { // Iterate through the columns in the square
                if (board[startRow + i][startCol + j] == c) return false; // If the value is
in the square, it can't be placed there and execution stops.
            }
        }
        return true; // If it passes all previous checks, then the value is valid and can be
placed there.
    }

    // Recursive DFS backtracking function to find all solutions and write them directly to
the output file stream
    bool solveRecursive(int row, int col, int& solutionCount, std::ofstream& outFile) {
        // If we have crossed the last row, a complete solution is found
```

```
        if (row == 9) { // Since the vectors are 0-indexed, 9 is out of bounds and reaching
it means that all cells have been successfully filled.
            solutionCount++; // All cells filled means the puzzle is solved, so increment the
counter.
            // Format and write each solution as a 9x9 grid of digits separated by spaces,
matching input format
            for (int i = 0; i < 9; ++i) {
                for (int j = 0; j < 9; ++j) {
                    outFile << board[i][j] << (j == 8 ? "" : " ");
                }
                outFile << "\n";
            }
            return false; // Return false to force backtracking and search for alternative
solutions
        }

        // Move to the next row if column reaches 9
        if (col == 9) { // Since the valid range of address is 0-8, 9 means that the columns
in the row have been successfully filled.
            return solveRecursive(row + 1, 0, solutionCount, outFile); // Go to the next row.
        }

        // If cell is already filled, skip to the next column
        if (board[row][col] != '_') {
            return solveRecursive(row, col + 1, solutionCount, outFile);
        }

        // Try digits '1' through '9'
        for (char c = '1'; c <= '9'; ++c) { // Hacky solutions taking advantage of ASCII
characters
            if (isValid(row, col, c)) { // If the value can be placed...
                board[row][col] = c; // Place that value
                solveRecursive(row, col + 1, solutionCount, outFile); // Recursively check
the next cell in sequence
                board[row][col] = '_'; // Backtrack
            }
        }

        return false; // If the code reaches this point, it has checked all possible
combinations.
    }

public:
    SudokuSolver() : board(9, std::vector<char>(9, '_')) {} // Formally declare the 2-D array
to be 9x9 filled with _.

    // Load puzzle from file
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename); // Open a file of the given name.
        if (!file.is_open()) { // If the file fails to open...
            return false; // Indicate that the file couldn't be loaded.
        }
        for (int i = 0; i < 9; ++i) { // Iterate through all 81 possible cells in the array
            for (int j = 0; j < 9; ++j) {
                file >> board[i][j]; // Iteratively writes non-whitespace characters into the
2-D array
            }
        }
```

```
        return true; // Indicate that the puzzle has been loaded into the program
successfully.
    }

    // Print the Sudoku board with clear grid formatting (preserved from Assignment 4 for
structure)
    void printBoard() const {
        for (int i = 0; i < 9; ++i) {
            if (i > 0 && i % 3 == 0) {
                std::cout << "------+-------+------\n";
            }
            for (int j = 0; j < 9; ++j) {
                if (j > 0 && j % 3 == 0) {
                    std::cout << "| ";
                }
                std::cout << board[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    // Driver method to process an individual puzzle file and generate the corresponding
solution file
    void solvePuzzleFile(const std::string& inputFilename, const std::string& outputFilename)
{
        // Error handling for missing or invalid input puzzle files
        if (!loadFromFile(inputFilename)) {
            std::cerr << "Error: Could not open input file " << inputFilename << ".\n";
            return;
        }

        // Open the corresponding output file
        std::ofstream outFile(outputFilename);
        if (!outFile.is_open()) {
            std::cerr << "Error: Could not create output file " << outputFilename << ".\n";
            return;
        }

        int solutionCount = 0; // Counter to hold the number of possible ways the puzzle can
be solved.
        solveRecursive(0, 0, solutionCount, outFile); // Solve the puzzle and write solutions
to file.

        // Output formatting requirements for summary lines
        if (solutionCount == 0) { // If no solutions, output exact required line
            outFile << "No solution found.\n";
        } else { // If at least one solution, include exact required line with total count
            outFile << "Solutions found: " << solutionCount << "\n";
        }

        outFile.close(); // Close the output file stream
    }
};

int main() { // Start of actually running the program
    SudokuSolver solver; // Instantiate the Solver class

    // Process puzzle1.txt through puzzle5.txt and generate solution1.txt through
solution5.txt
```

```
    for (int i = 1; i <= 5; ++i) {
        std::string inputFilename = "puzzle" + std::to_string(i) + ".txt";
        std::string outputFilename = "solution" + std::to_string(i) + ".txt";
        solver.solvePuzzleFile(inputFilename, outputFilename);
    }

    return 0; // Default return value.
} // End main
```